

#Naive Bayes Text Classification from Scratch

```
import re

import math

from collections import Counter, defaultdict
```

(١) تنظيف النص وتقطيعه (Tokenization)

```
def tokenize(text: str):
    """
    lowercase    تحويل النص إلى
    حذف علامات الترقيم
    إعادة النص كقائمة كلمات
    """
    text = text.lower()

    text = re.sub(r"[^\w\s]", " ", text)

    text = re.sub(r"\s+", " ", text).strip()

    return text.split()
```

(٢) Bag of Words (قاموس تكرارات)

```
def bow_counts(tokens):
    """
    {word: count}    تحويل قائمة كلمات إلى قاموس
    """
    return dict(Counter(tokens))
```

(٣) بناء المفردات Vocabulary

```
def build_vocab(docs):
    """
    اتحاد كل الكلمات في مستندات التدريب
    """
    vocab = set()
```

```

for text, _ in docs:
    vocab.update(tokenize(text))
return sorted(vocab)

(Prior probabilities P(c) حساب (٤ #
:
def compute_priors(docs)
    """
    عدد مستندات الفئة / عدد كل المستندات = P(c)
    """
    labels = [label for _, label in docs]
    counts = Counter(labels)
    n = len(labels)
    return {c: counts[c] / n for c in counts}

```

```

(٥ # حساب تكرارات الكلمات لكل فئة
:
def word_counts_by_class(docs)
    """
    bigdoc[c]: كل كلمات الفئة
    total[c]: عدد الكلمات الكلي في الفئة
    """
    counts = defaultdict(Counter)
    total = Counter

```

```

:
for text, label in docs
    tokens = tokenize(text)
    counts[label].update(tokens)
    total[label] += len(tokens)

    return counts, total

```

```

(٦ # التدريب Training

```

```
def train_nb(docs):
```

```
    """
```

تجهيز كل ما يلزم للتصنيف

```
    """
```

```
        vocab = build_vocab(docs)
```

```
        priors = compute_priors(docs)
```

```
        counts_c, total_c = word_counts_by_class(docs)
```

```
        return vocab, priors, counts_c, total_c
```

(٧) التصنيف باستخدام Naive Bayes + Log + Laplace

```
def classify_nb(text, vocab, priors, counts_c, total_c, classes=("pos", "neg")):
```

```
    """
```

تصنيف جملة جديدة

Vocabulary - تجاهل الكلمات غير الموجودة في

underflow لتفادي log - استخدام

```
    """
```

```
        tokens = tokenize(text)
```

```
        tokens = [t for t in tokens if t in vocab] # حذف OOV
```

```
        scores = {}
```

```
        V = len(vocab)
```

```
        for c in classes:
```

```
            score = math.log(priors[c])
```

```
            for w in tokens:
```

```
                pwc = (counts_c[c][w] + 1) / (total_c[c] + V) # Laplace smoothing
```

```
                score += math.log(pwc)
```

```
            scores[c] = score
```

```
prediction = max(scores, key=scores.get)
```

```
return prediction, scores, tokens
```

المقاييس Evaluation Metrics

```
def metrics_from_counts(tp, fp, fn, tn):
```

```
    accuracy = (tp + tn) / (tp + fp + fn + tn) if (tp+fp+fn+tn) else 0
```

```
    precision = tp / (tp + fp) if (tp+fp) else 0
```

```
    recall = tp / (tp + fn) if (tp+fn) else 0
```

```
    f1 = (2 * precision * recall) / (precision + recall) if (precision+recall) else 0
```

```
    return accuracy, precision, recall, f1
```

تقييم ثنائي كامل

```
:def evaluate_binary(docs, vocab, priors, counts_c, total_c, positive_label="pos")
```

```
    tp = fp = fn = tn = 0
```

```
:for text, gold in docs
```

```
    pred, _, _ = classify_nb(text, vocab, priors, counts_c, total_c)
```

```
:if pred == positive_label and gold == positive_label
```

```
    tp += 1
```

```
:elif pred == positive_label and gold != positive_label
```

```
    fp += 1
```

```
:elif pred != positive_label and gold == positive_label
```

```
    fn += 1
```

```
:else
```

```
    tn += 1
```

```
    return tp, fp, fn, tn, metrics_from_counts(tp, fp, fn, tn)
```

تشغيل المثال

بيانات التدريب

```
train_docs = [  
    ("just plain boring", "neg"),  
    ("entirely predictable and lacks energy", "neg"),  
    ("no surprises and very few laughs", "neg"),  
    ("very powerful", "pos"),  
    ("the most fun film of the summer", "pos"),  
]
```

تدريب النموذج

```
vocab, priors, counts_c, total_c = train_nb(train_docs)
```

مثال المحاضرة

```
pred, scores, used_tokens = classify_nb(  
    "predictable with no fun",  
    vocab, priors, counts_c, total_c  
)
```

```
print("Used tokens:", used_tokens)  
print("Scores:", scores)  
print("Prediction:", pred) # متوقع: neg
```

مجموعة اختبار

```
test_docs = [  
    ("fun film", "pos"),  
    ("boring and predictable", "neg"),  
    ("very fun", "pos"),
```

```

("no laughs", "neg"),
("powerful film", "pos"),
]

tp, fp, fn, tn, (acc, prec, rec, f1) = evaluate_binary(
    test_docs, vocab, priors, counts_c, total_c
)

print("\nTP, FP, FN, TN =", tp, fp, fn, tn)
print("Accuracy =", acc)
print("Precision =", prec)
print("Recall =", rec)
print("F1 =", f1)

```

الشرح المختصر الوافي

الكود يطبق Naive Bayes من الصفر بدون أي مكتبات ML. النص يُمثَّل باستخدام Bag of Words (تكرار الكلمات فقط).

يتم حساب:

Prior $P(c)$ من عدد المستندات

Likelihood $P(w|c)$ من تكرار الكلمات داخل كل فئة

نستخدم Laplace Smoothing لمنع الاحتمالات الصفرية.

نستخدم log لتجنب underflow وتحويل الضرب إلى جمع.

الكلمات غير الموجودة في التدريب (OOV) يتم تجاهلها.

مثال "predictable with no fun" يُصنَّف سلبي (neg) كما في المحاضرة.